The Performance Profiling of `pipe()`

Arjun Anil
University of Wisconsin–Madison

Aidan Liaw
University of Queensland

1 Introduction

The pipe is the most basic UNIX inter-process communication (IPC) mechanism: `pipe()` creates a uni-directional byte stream that, with `fork()`, connects two processes. In Linux it is a 64 KiB kernel buffer behind a mutex; reads on an empty pipe and writes on a full one block, and writes of at most `PIPE_BUF` (4096 B) are atomic [1]. We measure the latency and throughput of pipes for 4 B to 512 KB messages and use `perf`, `strace` and `ftrace` to find where the time goes.

2 Experimental Design

Variables. The independent variable is the message size, ten steps from 4 B to 512 KB that include `PIPE_BUF` (4096 B) and the pipe capacity (65536 B). The dependent variables are one-way latency, throughput and per-message counts (context switches, wakeups, cycles). Held fixed: pipe capacity, blocking I/O (polling would measure the loop), `SCHED_FIFO` priority 99, 4 GHz core frequency, and the parent, child and profilers on CPUs 0, 1 and 2 (P-cores).

Hypotheses. H1: variance between runs is small, given the isolation. **H2:** performance gets worse above `PIPE_BUF`, where we suspected writes might be split, and above the pipe capacity, where the producer must stall until the consumer drains the pipe.

Apparatus and timer. An Intel Core Ultra 7 258V laptop (four P-, four E-cores), Fedora 44, kernel 7.1.8, in its normal desktop session. A one-way message takes about a microsecond, so we compared four timers (smallest non-zero difference of back-to-back reads over 2000 trials) and checked each against a `sleep(2)` (Table 1). `gettimeofday()` and `CLOCK_MONOTONIC_COARSE` are too coarse. We time with `rdtsc`: its counter is invariant on this CPU and is read only in the parent, so both reads are on one core. It is calibrated against `CLOCK_MONOTONIC` (bracketed reads over 100 ms sleeps, minimum of 10 trials: 3.302 GHz). The fine timers agree within 10.5 μs in 2 s and read 36–47 μs high, the sleep overshooting; the independent `gettimeofday()` differs from `rdtsc` by 8.6 μs.

Table 1: Timer resolution and error after `sleep(2)`, CPU 0 at 4 GHz.

Timer	Resolution	Error
`gettimeofday`	1000 ns	+45.0 μs
`CLOCK_MONOTONIC`	11 ns	+46.9 μs
`rdtsc` (calibrated)	7.3 ns	+36.4 μs
`CLOCK_MONOTONIC_COARSE`	1 ms	+975.3 μs

Protocols. *Latency:* parent and child are joined by two pipes; the parent writes the message, the child echoes it, and the parent times the round trip. One-way latency is half of it, since two cores' counters cannot be compared. Per size: 20 warmup rounds, 2000 timed. *Throughput:* the parent streams 64 MiB in chunks of the given size (less for 4 B and 16 B: chunks capped at 2 million), the child returns one one-byte acknowledgement, and the timer runs from the first write to that acknowledgement; three transfers per run. The experiment is run five times; we report the minimum latency and best throughput, not the mean [2, 3], and use the runs to check reproducibility.

Profilers. They slow the benchmark, so they run separately and only counts and proportions are used: `perf stat` (counters [4]), `perf record` (every scheduler, IPI and idle tracepoint; cycles with call graphs), `strace` (system calls) and `ftrace` (kernel call tree [4]).

3 Results

A small message costs about 1 μs one way (0.98 μs at 4 B); latency is 1.14 μs at 1 KB, steps to 2.12 μs at 4 KB, and beyond the pipe capacity grows with the size (113.9 μs at 512 KB; Figure 1). Throughput peaks at 4,742 MiB/s at 16 KB and is 25% lower, 3,556–3,634 MiB/s, from 64 KB to 512 KB. The minimum latency reproduces within 0.9–9.7% across runs and the best throughput within 5.8–14% (29.5% and 39.2% for 16 B and 4 B, which move only 30.5 and 7.6 MiB). The 4 KB step is real: the 1st percentile doubles from 2.38 μs (1 KB) to 4.95 μs (4 KB).

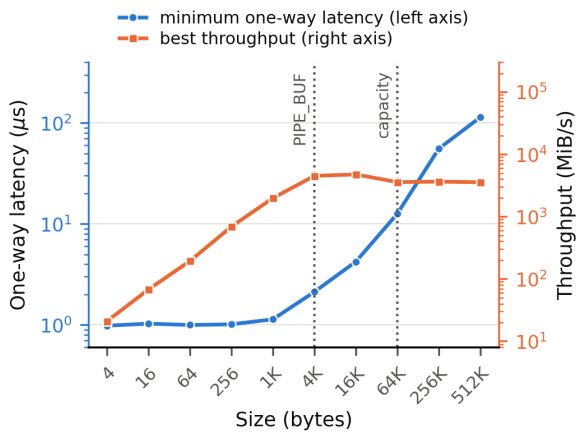


Figure 1: Minimum one-way latency (left axis) and best throughput (right axis) against message size, five runs. Dotted lines: `PIPE_BUF` (4 KB) and the pipe capacity (64 KB).

4 Where the Time Goes

Latency. `strace` shows one `write()` per message and $\lceil \text{size}/64\,\text{KiB} \rceil$ `read()`s at every size. A round trip of up to 64 KB costs 2 context switches, 2 wakeups and 2 inter-processor interrupts (IPIs); 256 KB and 512 KB cost 14 and 30 context switches ($2(2n-1)$ for n pipe capacities), since the writer sleeps until the pipe is drained. For 4 B the median round trip is 3.06 μs but the tasks execute for only 2.11 μs; in the other 0.95 μs (31%) neither runs, the time to wake a sleeping core. Traced, a wakeup takes 1.9–2.1 μs at every size: a fixed cost paid twice per round trip. As Figure 2 shows, no component dominates at 4 B (scheduling, wakeup and IPIs 37%, copying 7%), while at 64 KB and 512 KB copying is 66–68% and page allocation with cgroup accounting 11–13%.

Throughput. Streaming overlaps the two tasks: up to 16 KB there are almost no context switches and, from 64 B, both cores are nearly always busy (1.9–2.0 cores). From 64 KB each chunk costs 2, 8 and 16 context switches and only 0.86–0.91 cores are busy, though each KiB costs fewer cycles (991 against 1,741 at 16 KB): throughput falls 25%. Small chunks are limited by per-call cost (4 B: 43% locking, 30% syscall entry); at 4 KB, locking is 56%. The `ftrace` call trees agree (blocked `schedule` is 24% of kernel time).

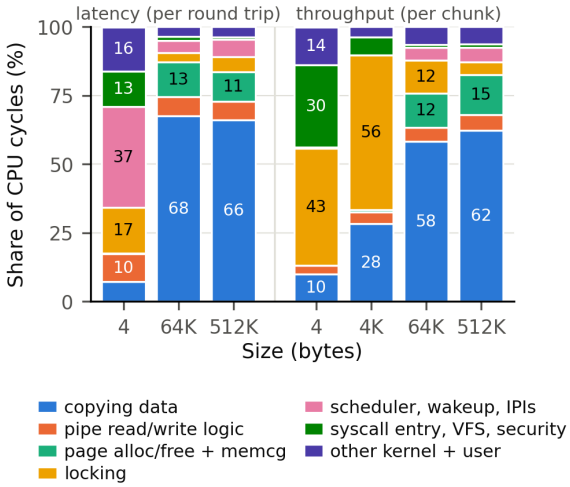


Figure 2: Share of CPU cycles by component for latency (per round trip) and throughput (per chunk). Kernel functions are grouped by name; labels give shares of 9% or more.

5 Discussion and Conclusion

H1 holds in part: minimum latencies reproduce within 10%, but the small-message median round trip (3.06 μs) is 50% above the minimum (1.96 μs) and 4 B/16 B transfers vary by 29–39%. **H2:** the `PIPE_BUF` half is not supported: every message is one `write()`, throughput peaks at 16 KB, and the 4 KB latency step has the same two wakeups as 4 B. That step is unexplained: the median round trip grows 2.1 μs from 1 KB to 4 KB, half CPU work and half time with neither task running, yet the traced wakeup path is size-independent. The capacity half is supported. Small messages are thus a scheduling problem, large ones a copying problem.

Limitations. One laptop, P-cores only, pinned at 4 GHz (unverified). The real-time scheduler adds bookkeeping (`cpupri_set` is 8.5% of 4 B cycles). The minimum is a best case; half a round trip assumes symmetry. Tracing perturbs the run, and capacity, page size and `PIPE_BUF` were not varied; sizes near 4096 B are the follow-up.

References

[1] Linux man-pages, *pipe(7)*. man7.org/linux/man-pages

[2] J. E. Smith. Characterizing computer performance with a single number. *CACM* 31(10), 1988.

[3] G. Heiser. Systems benchmarking crimes. UNSW web page.

[4] Linux perf wiki (perf.wiki.kernel.org) and kernel ftrace documentation (docs.kernel.org).